CS 2006 Operating Systems Spring 2026
Project Statement
BS - CS

General Instructions and Submission Guidelines

- **Integrity**
  Any form of plagiarism or sharing of solutions will result in zero marks for all involved parties. The use of AI tools to circumvent core concepts is strictly prohibited. All group members are expected to participate equally in the work.
- **Strict Deadlines**
  Late submissions will not be accepted under any circumstances. The deadline is final and will not be extended for technical or personal reasons. Submit early to avoid portal issues.
- **Directory Naming Conventions**
  Folders must be named as: **BCS Section_RollNumber_FullName** (e.g. BCS A_24i_0000_AliAhmed). Failure to follow this exact format results in an automatic 20%-mark deduction.
- **Deliverables and Structure**
  The final .zip file must contain all source files, a functional Makefile, and a PDF report including turnaround time analysis and gameplay screenshots.
- **Programming Standards**
  Maintain professional standards with well indented code, clear comments, and meaningful variable names. Proper interpretation of game requirements is part of the evaluation.
- **Personalized Roll Number Seed**
  You must use your specific Roll Number as the seed for all randomized elements. During the demo, you will be expected to explain how this seed influences your specific game performance.
- **Group Members:** A group of two members is allowed and preferred. Solo projects are discouraged, as they do not promote the development of teamwork skills. Groups consisting of more than two members are not permitted. Cross-section pairs are not permitted under any circumstances. For individual students, cases will be reviewed and

considered on a situational basis.

- **Docker Environment Requirement**

  - All submissions must be developed, compiled, and tested inside the provided Docker environment. This project relies on Linux-specific system calls (shared memory, signals, semaphores, pthreads), and therefore any solution not compatible with the Docker setup will not be considered valid.
  - Students are required to strictly follow the **Docker setup guide** provided separately. The final submission must compile and run successfully inside the container
  - **Note:** The required **folder structure, Dockerfile, and requirements.txt** must be included exactly as specified in the Docker guide. Any deviation from the prescribed structure may result in penalties during evaluation.

Deadline: 10 May 2026 (2359 PST)

Chrono Rift

1. Project Objective

The primary objective of Chrono Rift is to architect a multi process tactical game where a human player engages in combat against computer controlled entities. This game is modelled after turn-based combat mechanics found in classic role playing games. Students must build a robust environment where autonomous and non-autonomous entities compete for execution time based on dynamic attributes.

- **Process Isolation:** Every core component of the game must run in its own memory space to ensure fault isolation and game stability.
- **Systems Integration:** The project emphasizes the application of inter process communication and multi-threaded decision making logic.
- **Resource Coordination:** Students must manage complex data consistency using shared memory and internal memory based primitives.

2. Game Architecture and Role Isolation

The game architecture is designed as an asymmetric environment divided into three primary execution areas. The Game Arbiter process remains the central authority which manages the global state and enforces the rules of engagement.

- **Human Interfacing Process:** This is a separate process responsible for capturing user input and forwarding player actions to the Game Arbiter through shared memory. The Human Interfacing Process must be multi-threaded. A separate thread must be created for each player-controlled character. Player threads should read input from buffer when required. At any given time, only the thread corresponding to the currently active

player (as determined by the Arbiter) should process input. Other player threads must remain idle. The Human Interfacing Process must not directly modify the global game state. It only communicates player actions to the Arbiter, which is solely responsible for updating the game state.

- **Automated Strategic Process:** This area handles the decision making logic for all non-player characters or NPCs.
- **Threading Requirement:** This project must demonstrate explicit use of multithreading. Each NPC (enemy entity) must run in its own dedicated thread within the Automated Strategic Process. The system must support concurrent execution of multiple NPC threads, with proper synchronization when accessing shared resources. Implementations where multiple entities are handled sequentially within a single thread will not be accepted. Within the Automated Strategic Process, every individual computer controlled character must be managed by its own dedicated thread to simulate independent agents.
- **Lifecycle Management:** The game must gracefully handle the dynamic termination of processes when an entity is defeated. The Arbiter must update tracking queues and close relevant communication channels immediately upon a process exit.

3. Temporal Scheduling and Stamina Logic

The game utilizes a temporal scheduling model where a stamina attribute acts as the primary resource for execution. Every entity possesses a speed attribute that dictates the rate at which its stamina fills over time.

- **Arrival Time Logic:** Students must implement scheduling logic where the arrival time for an action is determined by the entity's maximum stamina and its speed. Each second the entity's speed is added to its current stamina. An entity may be scheduled for a move only when its stamina is fully filled.
- **Serial Execution:** While stamina accumulates concurrently for all characters, the execution of actions remains strictly serial. Only one character may perform an action at any given moment.
- **Action Commitment:** Only the first entity to reach a full capacity

threshold is permitted to act. Once the action is committed, that entity's stamina is depleted to zero and the scheduler recalculates the next execution window for all active participants.

4. Global State Synchronization and Memory

The integrity of the game relies on a global state that is shared across different process and thread boundaries. This state must be stored in a shared memory segment accessible to the Arbiter and all player processes.

- **Memory Based Primitives:** To prevent race conditions, students must implement a synchronization mechanism within the shared memory area.
- **Technical Constraint:** The Use of pipes (both unnamed and named) is not allowed. All inter-process communication must be implemented using shared memory. Synchronization must be handled using memory-based primitives such as mutexes or unnamed semaphores.
- **Data Consistency:** The state must remain consistent even when multiple NPC threads and the human player process attempt to read or write to the memory simultaneously.

5. Asynchronous Interrupts and the Stun Mechanic

The game must support an active time mechanic where the state of a process can be altered instantaneously by external events through asynchronous delivery.

- **The Stun Rule:** Certain high tier attacks can "Stun" a target. If a player or NPC is successfully stunned, their execution must be halted immediately for a duration of exactly 3 seconds.
- **Non-Blocking Interruption:** This interruption must occur asynchronously. The attacking process must deliver a message that forces the target process to pause its current logic without the target process actively checking a flag or a pipe. If the target's stamina was full, its turn should be skipped and should only be scheduled once the Stun period ends.
- **Recovery:** After the 3 second duration expires, the target process must

resume its logic from the exact point where it was interrupted, maintaining its previous stamina level.

6. Weapons and the Inventory Management Mechanic

The game includes a number of weapons that take up a certain number of slots in a player character's inventory.

If a player's inventory does not have enough contiguous slots to fit a weapon the player wants, existing weapons need to be swapped out to long term storage to make room.

- **Weapon Drops:** Weapons have a chance of being dropped when an enemy character is defeated, when a weapon drops, the player character has a choice to pick it up. If a player does not pick it up, an enemy is guaranteed to pick it up.
- **Inventory Space:** The player's primary inventory is a linear array of 20 slots. When a weapon is picked up, the game's space allocator must put the weapon at an appropriate free location with enough contiguous slots to hold the weapon.
- **Out of Space Scenario:** If there is not enough space for the weapon in the primary inventory, the space allocator must swap out some weapons from the primary inventory to long term storage and put the weapon in the freed up space. The allocator must only swap out as many weapons as are necessary.
- **Long Term Storage Retrieval:** Players may need to use weapons that have been swapped out to long term storage, to do this they can take the "Swap In" action and choose a weapon of their choice to bring back from the Long Term Storage. If there is no space in the inventory for this swapped in item, the same protocol from the Out of Space Scenario is to be used. Swapping In weapons costs a complete turn and the weapon cannot be used until the next turn.

7. Multi Resource Deadlocks and Artifacts

The game includes unique global artifacts that provide powerful buffs but require strict resource management.

- **The Artifact Rule:** There are two exclusive weapons in the game: **The Solar Core** and **The Lunar Blade**. To perform the Ultimate Ability, a character must simultaneously hold both items.
- **Resource Contention:** These items are limited. Only one instance of each artifact exists in the game world at any time. A process must "lock" an artifact before using it.
- **Dynamic Shared Artifact:** In addition to the predefined artifacts, a third artifact called the **Eclipse Relic** introduce dynamically at run time. When a character picks it up from the environment. Once introduced, it becomes part of the global artifact pool and follows the same exclusivity and locking rules as other artifacts.
- **Global Resource Table:** Students must maintain a shared resource table that tracks the state of all global artifacts (Solar Core, Lunar Blade, and Eclipse Relic if present). The table must indicate whether each resource is free or currently held by a specific entity.
- **Resource Access and Locking:** Before acquiring or releasing any artifact, a process must consult the shared resource table. Access to this table must be protected using proper synchronization mechanisms. When a process is modifying the table (acquiring or releasing a resource), it must lock the table to prevent concurrent updates. The table must be updated immediately after any change to ensure consistency.
- **Deadlock Scenarios:** If Player A locks the Solar Core and waits for the Lunar Blade, while NPC B locks the Lunar Blade and waits for the Solar Core, a circular wait occurs.
- **Mandatory Monitoring:** Students must implement a background logic thread within the Arbiter that detects these circular wait conditions. If a deadlock is detected, the Arbiter must force one process to release its held item to allow the game to proceed.

8. The Ultimate Ability Pause Mechanic

When a player character triggers the Ultimate Ability, the Automated Strategic Process must be fully suspended for a duration of 10 seconds before being resumed by the Arbiter. This mechanic introduces a high-impact tactical window that must be implemented with signals alone.

- **Signal-Only Enforcement:** The suspension and resumption of the Automated Strategic Process must be achieved exclusively through signals. Neither the Arbiter nor the Automated Strategic Process may use flags or pipes to coordinate this state transition.
- **NPC Turn Timeout:** To prevent the Arbiter from blocking indefinitely, a strict timeout policy is enforced on NPC turns. If the Strategic Process does not submit a move for its turn within 3 seconds, the Arbiter assumes that the chosen action was “Skip”.
- **Resumption Protocol:** The Arbiter must use SIGALRM and a custom handler to manage the suspension window. Once the window expires, the Arbiter must resume the Strategic Process with the updated staminas of all enemies.

9. Mandatory Graphical Interface and Rendering Thread

A real time user interface is a mandatory component of the project. The display provides a clear visualization of the underlying game mechanics to allow for easier debugging and evaluation.

- **Visual Representation:** The display must show real time stamina bars, health statistics, and an action log.
- **Asynchronous Rendering:** To ensure that the visualization does not interfere with the high precision timing of the scheduler, the rendering logic must be implemented in its own dedicated thread.
- **State Observation:** This rendering thread must read the current state directly from the synchronized shared memory region and update the display independently of the main scheduling loop.
- **Implementation:** The UI may be implemented as a GUI (using a GUI library of your choice e.g SFML, SDL, RayLib, GLFW) or as a TUI using ncurses. Basic CLIs are not allowed.

10. Gameplay Requirements and Configuration

This section defines all the concrete parameters governing all entities, items, and actions within the game. All values listed here are fixed and must be implemented exactly as specified.

- **Player Party:** At the start of the game, the player must be prompted to select the size of their party. The player may choose between 1 to 4 human-controlled characters. All human-controlled characters must be operated using the Human Interfacing Process defined in Section 2.
- **Enemy Configuration:** The number of concurrent enemies on screen at any given time is decided randomly on each run, the number must be between 2 and 9. Each NPC is managed by its own dedicated thread within the Automated Strategic Process as described in Section 2.
- **HP, Damage, Speed and Stamina:** Each entity must have random stats based on the following rules,
    - **HP** (Player Character): Your Roll No + a random number between 100 and 1000.
    - **HP** (Enemy): The last 2 digits of your Roll No + a random number between 50 and 200.
    - **Damage** (Player Character): Last digit of your Roll No + 10.
    - **Damage** (Enemy): Second last digit of your Roll No + 10.
    - **Speed** (Player Character): 100 / Number of Player Characters.
    - **Speed** (Enemy): Random number between 10 and 30.
    - **Max Stamina** (Player Character): 100
    - **Max Stamina** (Enemy): 150
- **Actions (Player):** Each player entity must perform exactly one of the following actions on each turn,
    - **Attack** (Strike): Reduces the selected enemy's HP by the player's damage stat and depletes the player's stamina to zero.
    - **Attack** (Exhaust): Reduces the selected enemy's Stamina by the player's damage stat and depletes the player's stamina to zero.
    - **Use Weapon**: Allows the player to attack using a weapon from their

inventory. Reduces the selected enemy's HP by the weapon's damage stat and depletes the player's stamina to zero.

- **Swap In**: Allows the player to swap in a weapon from their long term storage. Does not allow the player to use the weapon in the current turn. Depletes the player's stamina to zero.
- **Heal**: Restores 10% of the player's HP and depletes the player's stamina to zero.
- **Skip**: Skips the turn and depletes the player's stamina to 50%.

- **Actions (Enemies):** Each enemy entity must perform exactly one of the following actions on each turn,
  - **Attack** (Strike): Reduces a chosen player's HP by the enemy's damage stat and depletes the enemy's stamina to zero.
  - **Skip**: Skips the turn and depletes the enemy's stamina to 50%.
- **Weapon Inventory and Damage Table:** The player's primary inventory is a linear array of 20 slots as defined in Section 6. The following weapons are available in the game world. A player may hold copies of the same weapon. If an NPC holds a weapon, the weapon will not be dropped when it dies. Slot size refers to the number of contiguous inventory slots the weapon occupies.

Weapon Name	Slot Size	Damage Output
Solar Core	10	95
Lunar Blade	10	90
Iron Halberd	7	55
Venom Dagger	4	30
Thunderstaff	6	50

Weapon Name	Slot Size	Damage Output
Obsidian Axe	5	45
Frostbow	6	48
Splinter Stick	2	12

The Solar Core and Lunar Blade each occupy 10 slots, meaning a character holding both simultaneously has no remaining inventory space for any other weapon. This is by design and is a hard constraint the space allocator must enforce. The Splinter Stick's 2-slot footprint is intentionally narrow and will produce fragmentation edge cases that the allocator must handle correctly.

- **Ultimate Ability Eligibility:** A player character may only trigger the Ultimate Ability if both the Solar Core and the Lunar Blade are present in their active primary inventory simultaneously, as described in Sections 7 and 8.

- **Game Completion:** The game completes and exits gracefully when either,

    - All Player Characters die (Lose Condition)

    - The Players kill a total of 10 enemies (Win Condition)

    - The Player chooses to quit for which the Arbiter is sent a SIGTERM by the Human Interfacing Process (Quit Condition).

11. Bonus Multiplayer Extension

The final implementation must focus on the stability and logical correctness of the underlying OS concepts. This ensures the project remains a technically sound game rather than just a visual simulation.

- **System Performance:** Students will be evaluated based on the stability of their multi process architecture and the accuracy of their scheduling logic under high load.
- **Accuracy of Execution:** The game should be able to prove that stamina depletion and action commitment follow the mathematical arrival time logic described in Section 5.
- **Multiplayer Bonus:** As an optional Bonus challenge, students may implement a Local Multiplayer Mode where two separate human controlled processes compete against each other or against a shared pool of NPCs.

Reference Material:

The following videos are provided to help you get a general idea of the project and overall inspiration:

- https://youtu.be/OQtv2KEGvsw?si=UP6k7p7he7J0vpLA&t=74
- https://youtu.be/33AKougRwew

These resources are only for understanding the basic gameplay and overall environment setup. Students must strictly follow the requirements and specifications defined in this project statement while implementing their solution.